Temco Platform

The catalog · built Nov 12, 2026

The catalog

The platform's memory: one SQLite file, owned by `temco-store`, at `<globalStorageUri>/catalog/catalog.db` (with its WAL beside it). Every module reads it through `Catalog.open(context)`; no module opens its own database.

A row is written by an act — a read, a create, a build — and read by a surface. Nothing in the file is derived from the network at read time: a list from GitHub, for example, is a record of what GitHub *said*, with the time it said it.

Shape (schema version 6)

Table	What it holds
`meta`	`schema_version`, and nothing else.
`organisations`	Where repositories come from: `kind` (`organisation` | `account` | `repository`), `name`, unique `slug`, `url`, `added_at`, `last_read`, `state` (`read` | `refused` | `named`).
`repositories`	The rows of each list, `org_id` → `organisations(id)` **on delete cascade**, keyed by `(org_id, name)`. A cache of what GitHub last answered — replaced whole per organisation, kept when a read is refused. Indexed by `(org_id, pushed_at DESC)`.
`sources`	Where content actually comes from: a git remote at a branch, a folder, or an archive. Unique `slug`; `last_read`, `last_read_mode`, `last_commit`. **The truth about a repository is a source, not a cached row.**
`documents`	The projects a person manages — `name`, unique `slug`, `version`, `audience`, `owner`, `origin`, `template`, timestamps.
`doc_sources`	Many-to-many: a document cites sources, each with its own `branch`, `include_glob`, `role`.
`recipes`	One row per document: the engine (`docfx`), its `config_path`, `script`, and the `outputs` it must produce.
`builds`, `artifacts`	A build attempt and what it left behind, by document.
`commits`, `commit_pages`	What a read found in a source's history, and which pages a commit touched.
`pages`	The pages a read recorded, by source and page id.
`scans`	One row per read: mode, revision, counts.
`events`	What the platform noticed and has not shown (`seen_at` null).
`drafts`	A page's text with no file behind it: `document_slug`, `page_id` (the id it *will* have), `title`, `kind`, `body`, `state` (`draft` | `accepted` | `discarded`).
`releases`	Published versions, by document.

Versioning

`SCHEMA` is applied with `IF NOT EXISTS` on every open, then `meta.schema_version` decides whether migrations run. A version from the future is **refused in words** rather than half-read. `MIGRATIONS[n]` is

what a version `n-1` file needs to become `n`:

- 4 — `documents.origin`, `documents.template`
- 6 — drops `repo_lists`, the one-turn JSON blob that `organisations` + `repositories` replaced

Where the settings sit

Three settings decide what the repositories surfaces act on, and all are written by `temcoRepositories.settings` (the page) or by VS Code's own settings JSON:

- `temcoRepositories.organisation` — the enterprise the dashboard opens on, and the one seeded into `organisations` the first time the page opens.
- `temcoRepositories.workspaceFolder` — the root of the working area. Clones go to `source/` under it (one copy per repository, shared by every project that cites it). Empty means `~/temco`. It is outside every repository on purpose: nothing is ever written to a source, and nothing is pushed.
- `temcoRepositories.projectsFolder` — where a project's own folder is made, `<this>/<slug>/`. Empty means the working area above, which is where every project went before the setting existed; pointing it at a folder of its own keeps projects apart from the shared clones.